

LITETOPK: EXPLOITING THE CURSE OF DIMENSIONALITY FOR A FUSED INDEXER-TOPK KERNEL IN LONG-CONTEXT SPARSE ATTENTION

Ziqi Yin

Nanyang Technological University

Jianyang Gao

ETH Zurich

Peiqi Yin

The Chinese University of Hong Kong

Jiangneng Li

Nanyang Technological University

Gao Cong

Nanyang Technological University

ABSTRACT

Indexer-TopK, the operation to compute the scores and select the top- k candidates, is widely used by sparse attention algorithms in large language models and vector retrieval in recommendation systems and vector databases. However, existing GPU-based Indexer-TopK kernels like DeepSeek Sparse Attention (DSA) remain inefficient due to excessive global memory traffic, costly synchronization, and prohibitive memory overhead. In this study, inspired by the curse of dimensionality phenomenon, we first observe that sparse attention scores exhibit a score concentration phenomenon, where scores tend to fall within a narrow range. Based on this observation, we propose LITETOPK, an efficient fused Indexer-TopK kernel. LITETOPK first samples a small subset of data to estimate query-data score ranges, then partitions candidates into bins accordingly. This organization allows the LITETOPK kernel to maintain a tight approximate threshold online, write back only promising candidates, reduce unnecessary I/O and memory overhead while preserving exact Top- k correctness. Building on LITETOPK, we further propose LITEDSA, which exploits the similarity of top- k candidate sets among neighboring tokens. LITEDSA packs neighboring tokens' candidates for joint computation and masks out extra scores for each query, thereby reducing memory traffic while preserving correctness. Experimental results in a real-world deployment environment with eight B200 GPUs show that LITETOPK+LITEDSA accelerates the prefill stage of GLM 5.2 by $1.34\times$, with no performance loss and lower memory overhead.

1 INTRODUCTION

Indexer-TopK, which computes scores under a specific scoring function and selects the per-row top- k elements, has become a core primitive in modern machine learning systems, with applications in large language model (LLM) inference (Tang et al., 2024; Liu et al., 2025; Synk et al., 2025), recommendation systems (Khandagale et al., 2025; Gao et al., 2021), and information retrieval (Lee et al., 2023). Representative examples include:

1. In long-context LLM inference, attention computation becomes prohibitively expensive during the prefill stage as contexts grow to millions of tokens, motivating sparse-attention systems (Ribar et al., 2024; Chen et al., 2024; Liu et al., 2025; Synk et al., 2025; Tang et al., 2024) such as DeepSeek Sparse Attention (DSA) (DeepSeek-AI, 2026) to approximate full attention using only a small subset of tokens. Specifically, during the prefill stage, DSA treats the current text chunk as queries and the historical context as candidate keys and values. It computes and stores lightweight relevance scores between the preceding context and the chunk text, then applies top- k selection to identify the most relevant historical tokens for constructing sparse attention. While this design reduces the cost of full attention, its score-computation and selection pipeline itself becomes a major performance bottleneck. As shown in Figure 1, the DSA kernel, which includes both score

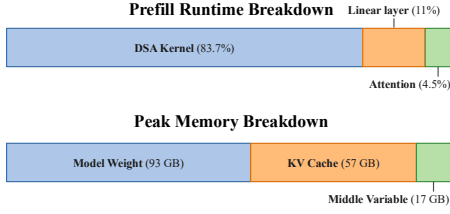


Figure 1: Breakdown of GLM-5.2 Prefill Runtime and Peak Memory Usage Across 8 B200 GPUs Using 8-Way Tensor Parallelism and Expert Parallelism.

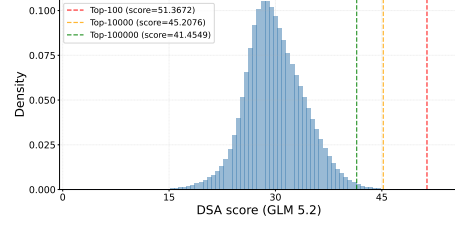


Figure 2: DSA Score distribution of GLM 5.2’s first layer during prefill on the Wiki dataset.

computation and Top- k selection, dominates the prefill runtime of GLM 5.2 (GLM-5-Team et al., 2026) at a 1M context length, accounting for 83.7% of the total runtime.

2. Recommendation systems and information retrieval systems typically encode queries/users and candidate items into high-dimensional embeddings using deep learning models (Khandagale et al., 2025; Gao et al., 2021). During candidate retrieval, they use Indexer-TopK to identify the top- k candidates with the highest similarity scores, where k can be on the order of thousands. The retrieved candidates are then re-ranked by more sophisticated models.

Despite the broad applications, existing Indexer-TopK implementations remain inefficient on GPUs because they typically decouple score computation and selection into two stages: first, computing and materializing the full score matrix in high-bandwidth memory (HBM) and then performing a top- k selection, such as radix-select (Alabi et al., 2012). However, writing the entire score matrix back to HBM introduces two major overheads. First, it incurs substantial **memory overhead**. For example, the DSA kernel incurs a runtime memory overhead of 32 GB under a 1M-token prefill with a moderate chunk size (e.g., 8192 in vLLM Kwon et al. (2023)). This is because the kernel needs to keep the full score matrix before the top- k selection. Such runtime memory overhead reduces the portion of HBM for storing the KV cache, which is already tight in existing LLM serving systems. As shown in Figure 1, deploying GLM-5.2 on 8 B200 GPUs with 0.95 memory utilization limit provides 170 GB per GPU, while the model weights, KV cache, and intermediate activations together consume 167 GB, leaving less than 3 GB of headroom. In practice, this forces vLLM (Kwon et al., 2023) to further split the 8192-token chunk into smaller sub-chunks during prefill, such as 128, in order to reduce the HBM footprint and avoid out-of-memory errors, but at the cost of lower prefill throughput¹. Second, it incurs significant **latency overhead**, since the score matrix must be written back to HBM and subsequently read multiple times during Top- k selection, leading to significant latency. The same issue is also noted in the vLLM blog², which observes that “a clear challenge is that at high batch size with long context, the logits tensor is materialized before running a row-wise top- k .”

To address this issue, we study the distribution of the scores in sparse attention and observe that these scores consistently show a concentration phenomenon, which is brought by the curse of dimensionality in a high-dimensional space (Weber et al., 1998; Indyk & Motwani, 1998). In particular, in high-dimensional vector spaces, similarity scores between vectors, such as Euclidean distance and inner product, tend to concentrate within a narrow range while exhibiting an extremely long-tailed distribution, a phenomenon widely observed in real-world datasets (Yin et al., 2026; Bruch, 2024). Since sparse-attention scores are typically derived from the similarity scores between high-dimensional vectors, this concentration effect also appears in sparse attention. For example, DSA applies ReLU-based weighting to inner-product scores, resulting in a distribution that exhibits the same narrow-range concentration and long-tail behavior. As shown in Figure 2, for the first layer of GLM-5.2 under a 1M-token context, DSA scores are concentrated between 25 and 35, while scores greater than or equal to 45 are extremely rare, with only about 10K such scores.

Leveraging this pattern, we propose LiteTopK, a kernel that follows a sample-filter-select framework. It initializes a tight threshold and dynamically updates it to filter out unpromising candidates, effectively reducing memory overhead and I/O cost. Specifically, in the sample phase, LiteTopK exploits

¹<https://github.com/vllm-project/vllm/pull/36178>

²<https://vllm.ai/blog/2025-09-29-deepseek-v3-2>

the observation that, in sparse attention, adjacent tokens attend to the same prefill tokens and often carry related semantics, making their score distributions likely to be similar. Therefore, we use the top- k' scores from the previous chunk ($k' = 3k$ for DSA), as a lightweight sample to estimate the score distribution of the current chunk. Then, we apply equal-width quantization to partition the estimated score range into non-overlapping sub-ranges, referred to as bins, initialize a histogram to count the number of candidates in each bin, and identify the bin containing the local Top- k element as the threshold bin.

In the filtering phase, we compute scores for all candidates and dynamically maintain a threshold bin. The distance upper bound of this bin is then used to filter out unpromising candidates, thereby substantially reducing write-back pressure. To prevent histogram updates from slowing down the overall GPU-parallel execution, we assign idle warps to update the histogram via overwrite writes instead of lock-based operations, since a stale and slightly loose threshold does not affect correctness or efficiency. To minimize the overhead of computing bin IDs, we replace the original DSA-score computation with bin-ID computation without introducing additional latency, directly store the intermediate bin IDs in floating-point format, and restore the corresponding scores only when returning the final results. To reduce write overhead, we maintain a list in warp-local shared memory and flush it in batches. In the selection phase, selection is required only within the threshold bin, whereas candidates from all preceding bins are directly written to the output, thereby substantially reducing I/O cost and selection overhead.

Building on LITETOPK, we further developed LITEDSA. We observed that in sparse attention, neighboring tokens often share highly similar top- k candidates, while the matrix multiplications involved in sparse attention are relatively small and the overall workload is primarily memory-bound. Based on this insight, LITEDSA merges and packs the top- k candidates of neighboring tokens, allowing these neighboring tokens to attend jointly over the combined candidate set. After the attention computation, LITEDSA masks out the extra scores that are irrelevant to each individual token, ensuring that the final output remains exactly identical to the original sparse attention result. In this way, LITEDSA significantly reduces redundant memory I/O without changing the result, making it particularly effective for memory-bound sparse attention computation.

In summary, this work makes the following contributions:

- We are the first to observe and characterize the distance concentration phenomenon in sparse attention, trace its origin to the well-known curse of dimensionality in high-dimensional spaces, and further exploit this property to improve sparse attention kernel design. We further observe and leverage the neighbor-similarity pattern among adjacent tokens, and develop LITEDSA to further accelerate sparse attention computation.
- We propose LITETOPK, the first Indexer-TopK fused kernel, which substantially reduces write-back memory pressure and achieves a $1.27\times$ speedup over the fastest existing DSA kernel. Building on LITETOPK, we further propose LITEDSA, which substantially accelerates the DeepSeek sparse attention computation by up to $XX\times$. Notably, the design principle of our kernel can also be extended to other sparse attention kernels.
- In a real-world deployment on 8 B200 GPUs, LITETOPK achieves a $1.22\times$ end-to-end speedup on GLM-5.2's prefill stage while using less memory. Building on LITETOPK, LITEDSA further improves the end-to-end prefill speedup to $1.39\times$. The source code of our implementation is available at <https://github.com/Heisenberg-Yin/LiteTopK>.

2 BACKGROUND

2.1 SPARSE ATTENTION

As million-token context windows are increasingly adopted in real-world LLM production environments, sparse attention algorithms (Lai et al., 2026; Yuan et al., 2025; Liu et al., 2025; DeepSeek-AI, 2026; Tang et al., 2024; Ribar et al., 2024) has become indispensable for reducing the I/O and computational costs of full attention. A representative example is DeepSeek Sparse Attention (DSA) (Liu et al., 2025), a natively trained sparse attention module, which has become the foundation for massively deployed models such as DeepSeek-V4 (DeepSeek-AI, 2026), GLM (GLM-5-Team et al., 2026), and Longcat Meituan LongCat Team (2026). DSA follows a score-then-select paradigm. It

develops a lightweight indexer, which stores an indexer key vector $k_s \in \mathbb{R}^d$ for every preceding token s . For a query token t , it derives H indexer heads $q_{t,j} \in \mathbb{R}^d, j = 1, \dots, H$ and aggregates their head-wise scores to compute the index score as follows:

$$I_{t,s} = \sum_j^H w_{t,j} \text{ReLU}((q_{t,j})^\top k_s). \quad (1)$$

where $w_{t,j}$ is a query-dependent weight used to aggregate the per-head scores. A per-query top- k selection with $k = 2,048$ is then applied to the aggregated score $I_{t,s}$, and full attention is computed over the Key/Value vectors of the selected top-2,048 tokens.

Another class of sparse-attention methods is training-free, meaning that they can be directly applied to off-the-shelf LLM checkpoints without modifying model weights or requiring sparse-attention pretraining Yang et al. (2025b;a); Li et al. (2024). These methods typically follow the same design: performing score-then-select independently for each attention head. They mainly differ in how per-head relevance scores are computed, ranging from exact query-key inner products (Gupta et al., 2021; Yang et al., 2025a) to approximate scores derived from reduced representations, such as salient query channels (Ribar et al., 2024) and product quantization (Zhang et al., 2025). However, these training-free methods typically introduce non-negligible accuracy degradation Nawrot et al. (2026); Yuan et al. (2025). In this study, we integrate LiteTopK with DSA, a representative natively trained sparse-attention method, while LiteTopK is broadly applicable to other score-then-select sparse-attention paradigms as well.

2.2 LLM SYSTEM

To support the training and inference of large language models, LLM systems have become essential infrastructure, with representative systems including vLLM Kwon et al. (2023), SGLang Zheng et al. (2024), and FlashInfer Ye et al. (2025). These systems aim to improve efficiency, including throughput and latency, as well as GPU memory utilization, while preserving algorithmic correctness. Existing efforts can be broadly categorized along four main dimensions: (1) reducing GPU memory traffic, as in FlashAttention Dao et al. (2022), which tiles attention and applies online softmax so that the attention score matrix is never materialized in HBM; (2) increasing parallelism, as in FlashDecoding Hong et al. (2024), which splits the KV dimension across CTAs to better saturate GPU SMs when the number of queries is small; (3) overlapping computation with data movement, as in FlashAttention-3 Shah et al. (2024), which uses warp-specialized pipelines and asynchronous copies to hide memory-access latency; and (4) reducing redundant memory overhead, as in PagedAttention Kwon et al. (2023) and RadixAttention Zheng et al. (2024), which respectively manage KV cache with paged memory allocation and reuse shared prefix states across requests. Despite the importance of GPU memory management, to the best of our knowledge, no prior LLM serving system has explored fusing indexer scoring with top- k selection to avoid materializing intermediate scores in HBM. To fill this gap, we propose LiteTopK, which not only reduces the latency but also significantly reduces memory overhead.

Very recently, Yang et al. (2026) proposed an more I/O-efficient KNN kernel, which fuses similarity scoring with top- k selection by maintaining local top- k set within each CTA and reduces these partial results to the final top- k set, thus avoiding full score-matrix writes to HBM. However, this approach achieves high efficiency only for very small k values, such as 10, and degrades sharply at larger k values due to limited per-CTA register capacity and costly synchronization. This is confirmed by our experiments, where its efficiency drops sharply as increases from 128 to 1,024 (Section 4.2). This limits its applicability, as sparse attention methods such as DSA often require retrieving thousands of entries, e.g., 2,048.

2.3 GPU ARCHITECTURE

Parallelism is fundamental to high-performance GPU computation. One source of such high-performance comes from the heterogeneous computational units: modern NVIDIA GPUs use Tensor Cores for matrix-matrix multiplication and CUDA cores for scalar and element-wise operations. Since these units have separate pipelines and can execute concurrently, per-score scalar operations can overlap with Tensor Core score computation. For example, in DSA, the dot products are computed on Tensor Cores, while the ReLU and weighted aggregation are performed on CUDA cores (see

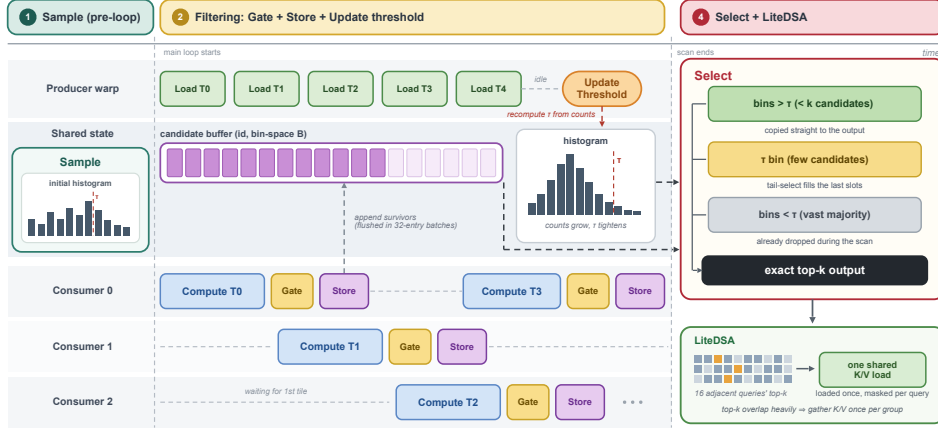


Figure 3: Illustration of the LiteTopK and LiteDSA.

Equation 1). Another source is thread-level parallelism: GPU kernels are organized into Cooperative Thread Arrays (CTAs), each containing multiple warps. Starting with the Hopper architecture, different warps within a CTA can be specialized for distinct roles: producer warps stream data tiles from HBM into shared memory, while consumer warps compute on previously loaded tiles. This forms a software pipeline that overlaps data movement with computation (see Figure 3). LITETOPK exploits these pipeline gaps to hide its additional operations.

3 METHODOLOGY

3.1 PROBLEM DEFINITION AND OVERVIEW

Problem Definition. Given a query set Q and a candidate set X , our goal is to exactly identify, for each query, the k candidates with the highest scores under a scoring function f :

$$\text{TopK}(q) = \arg \text{topk}_{x \in X} f(q, x), \quad q \in Q. \quad (2)$$

Sparse attention, where f is an indexer score such as the DSA score in Equation 1, is the focus of this paper, while we also consider other scenarios such as k -nearest-neighbor (k -NN) search.

Overview. Figure 3 presents a CTA-level overview of our proposed LITETOPK and LITEDSA methods. Specifically, LITETOPK follows a sample-filter-select workflow. Before the main loop starts, LITETOPK performs the sample phase: it samples a small subset of the dataset, partitions the sampled scores into equal-width bins, and identifies the bin containing the local top- k threshold, referred to as the threshold bin (τ). Since the global top- k threshold is guaranteed to be no lower than the local top- k threshold, filtering out candidates whose scores fall below τ ’s lower distance preserves correctness.

After the sample phase, LITETOPK enters the main loop and performs filtering in a fused manner with the existing indexer kernels. In particular, modern indexers typically follow a warp-specialized pipeline, as detailed in Section 2, where different warps within a CTA collaboratively handle memory loading and score computation, as shown in Figure 3. LITETOPK implements filtering as a lightweight integer gate embedded into the existing indexer pipeline. The lightweight gate is evaluated on the CUDA core where scores are computed and therefore does not interrupt the original pipeline execution. Then the passing candidates are appended into the local queue in the shared memory, as shown in Figure 3, which are flushed in batches to HBM later. Regarding the refresh phase, we periodically use the idle warps to recompute the threshold bin from the histogram, so the gate tightens as the scan proceeds.

In the selection phase, candidates in bins above the final threshold bin are written directly to the output, and a tail selection is performed only within the threshold bin. Meanwhile, based on the selected top- k indices, we efficiently derive the union set of the top- k candidates from neighboring tokens along with the corresponding mask bitmap, and then pack these candidates to enable batched matrix

multiplication, thereby losslessly accelerating sparse attention computation. Next, we introduce our proposed LITETOPK and LITEDSA in detail.

3.2 LITETOPK

We now detail the four stages in turn: sample, filtering, select and

Sampling. The sample serves two purposes: it must span the score range so that the bins are well scaled, and it should contain scores close to the true top- k threshold so that the initial gate is tight. An overly coarse sample degrades the effectiveness of filtering. For sparse attention, we exploit the observation that prefix keys and values are shared across chunks, and that adjacent chunks naturally exhibit similar semantics and therefore tend to attend to the same prefix tokens. We therefore reuse the k' tokens (e.g., $k' = 3k$ for DSA) that occur most frequently in the top- k results of the previous chunk as the sample for the current chunk. Such tokens are likely to remain high-scoring, yielding a tight initial threshold. Since a prefill chunk contains thousands of queries (e.g., 8,192 tokens), the cost of selecting the top- k' tokens is amortized over thousands of scoring computations, and the selection is executed asynchronously, making it negligible in practice. For k -NN search, whose score distribution is also concentrated but exhibits no such temporal structure, random sampling suffices.

We then score the k' sampled candidates to obtain the minimum and maximum sample scores $s_{\min}$ and $s_{\max}$, and apply equal-width quantization to partition $[s_{\min}, s_{\max}]$ into m bins of width $\Delta = (s_{\max} - s_{\min})/m$: a score s falls into bin $\lfloor (s - s_{\min}) \cdot \delta \rfloor$. Here, we store the reciprocal $\delta = 1/\Delta$ so that bin IDs are computed with a multiplication rather than a division, which is more efficient on GPU. The histogram is initialized with the bin counts of the sample, and the threshold bin is initialized as the bin containing the k -th largest sampled score. During filtering, a candidate passes the gate if and only if its bin ID is no smaller than the threshold bin ID, i.e., its score is at least the lower edge of the threshold bin. This initialization is conservative: since the sample is a subset of X , its k -th largest score cannot exceed the k -th largest score over all of X , so the true top- k threshold lies in or above the threshold bin and every true top- k candidate passes the gate.

Filtering. Filtering is fused into the existing scoring pipeline without changing its scoring semantics, and involves two concerns: computing bin IDs at negligible cost, and writing back the passing candidates efficiently. For the first concern, consider how scores are produced. When scores are produced directly by Tensor Cores, as in inner-product-based sparse-attention kernels Yang et al. (2025a), the bin ID is computed on CUDA cores immediately after each score is produced, adding only a few scalar instructions per score (an FFMA for the bin ID and a comparison for the gate). Since such kernels are Tensor-Core-bound and their CUDA cores are largely idle (Section 3.1), this post-processing overlaps with the matrix multiplication of subsequent tiles and introduces negligible overhead. The kernel simply outputs each score together with its bin ID. DSA is more challenging: its score is finalized on CUDA cores, so any additional arithmetic there directly adds latency. We therefore fold the bin mapping into the score computation itself. The key observation is that the bin-space score $B_{t,s} = (I_{t,s} - s_{\min}) \cdot \delta$ is an affine transform of the DSA score, and can be produced by the existing computation:

$$B_{t,s} = (I_{t,s} - s_{\min}) \cdot \delta = \sum_j w'_{t,j} \text{ReLU}(q_{t,j})^\top k_s - s_{\min} \cdot \delta, \quad w'_{t,j} = w_{t,j} \cdot \delta. \quad (3)$$

Since δ remains fixed throughout the scan, the weights are pre-scaled ($w'_{t,j} = w_{t,j} \delta$) once, before being loaded into registers. Moreover, the weighted sum is already computed with a chain of FFMA instructions whose accumulator is normally initialized to zero. We simply initialize it to the affine offset $-s_{\min} \cdot \delta$ instead, which is effectively free. The FFMA chain then directly outputs $B_{t,s}$, from which the bin ID follows, adding no instructions to the critical path. For each surviving candidate we store the floating-point bin-space score $B_{t,s}$ and discard the raw score. Because the transform is a fixed invertible affine map, the raw score can be recovered from $s_{\min}$ and δ at the final output stage, making the transformation lossless.

The second concern is write-back. Candidates that pass the gate are not written to the shared-memory candidate buffer immediately. Instead, each warp first collects them in a small unordered staging list and flushes the list only when it reaches a predefined length, such as 32 or 64 entries, using all 32 threads of the warp concurrently. This batching has two benefits. First, it reduces atomic contention on the buffer tail: the tail pointer is updated once per flush rather than once per candidate, cutting the

contended updates to $1/32$ or $1/64$ of the original frequency. Second, it better utilizes warp-level parallelism: all 32 lanes participate in a batched flush, whereas immediate per-candidate writes activate a single lane and leave the rest idle. In addition, each passing candidate issues one atomic increment to its histogram bin. These increments are fire-and-forget: their results are consumed only by the periodic threshold refresh described next, never on the critical path, so they do not block the main execution flow.

Threshold Refresh. As the histogram grows, the gate can tighten beyond its initialization. Periodically, an idle warp, typically the producer after it has issued all fetch commands, or any other naturally idle warp, recomputes the threshold bin by accumulating histogram counts from the highest bin downward until the cumulative count reaches k , and publishes it with a plain overwrite write instead of an atomic update. This requires no synchronization and therefore preserves the existing parallel execution order on the GPU. A stale threshold merely loosens the gate, admitting a few extra candidates, but are rare under score concentration. Threshold maintenance thus stays entirely off the critical path, hidden in idle cycles.

Top- k Selection. When the scoring process completes, all surviving candidates reside in the candidate buffer together with their bin-space scores, and the final threshold bin is known. By the refresh rule, the bins above the threshold bin jointly contain fewer than k candidates. These candidates are written directly to the output. The remaining slots are filled by a tail selection restricted to the candidates inside the threshold bin, whose number is small under score concentration. Raw scores are recovered from the stored bin-space values via the inverse affine map. Compared with decoupled designs that run a full top- k selection, such as radix select, over all $|X|$ materialized scores, LiteTopK selects among only the threshold-bin candidates, which substantially reduces the selection cost.

3.3 LITEDSA

The key idea of LITEDSA is to avoid letting each token scan its own Key/Value vectors independently. Instead, it merges the KV vectors of adjacent query tokens into a larger shared scan window. This lets the adjacent block reuse the same loading and compute pipeline, improving throughput and hardware efficiency. The tradeoff is that the shared window may include some KV positions that are valid for only some rows but invalid for others.

To address this, LITEDSA constructs the union candidate set with a shared-memory bitmap and derives a compact per-token membership mask. For each token group, a CTA first clears a bit-packed bitmap over the candidate position space. During the LITETOPK selection phase, when a candidate is selected, the corresponding lane marks its entry in the idmap by issuing a parallel atomicOr update, so candidates selected by multiple tokens are naturally deduplicated into the same union set. After insertion, the bitmap is swept and compacted into a single explicit physical KV list, which is shared by all tokens in the group. In a second pass, LITEDSA builds a query-major membership mask, where each query owns a bitmask over the compacted union slots, indicating which union entries actually belong to that query’s original top- k set. During attention, all tokens within the same group compute over the shared list, but each row ANDs the valid-lane mask with its own membership mask and sets non-member scores to $-\infty$. Thus, LITEDSA uses a bitmap to deduplicate and compact the group-level union, while using per-query bitmasks to preserve exact membership semantics without storing a separate explicit candidate list for every query.

4 EXPERIMENTS

4.1 EXPERIMENTAL SETUP

Evaluation Overview. We organize our evaluation around two application domains. For long-context large language model serving, we first evaluate LITETOPK and LITEDSA in an end-to-end deployment of the prefill stage of GLM-5.2 and LongCat 2.0, which is the primary bottleneck in long-context serving Bai et al. (2026). We then isolate the sparse-attention kernel to characterize their latency and memory consumption under controlled configurations. For large-scale vector retrieval, we evaluate whether the benefits of LITETOPK generalize beyond sparse attention workloads. Together, these experiments assess the application-level effectiveness, kernel-level efficiency, and cross-domain generality of LiteTopK.

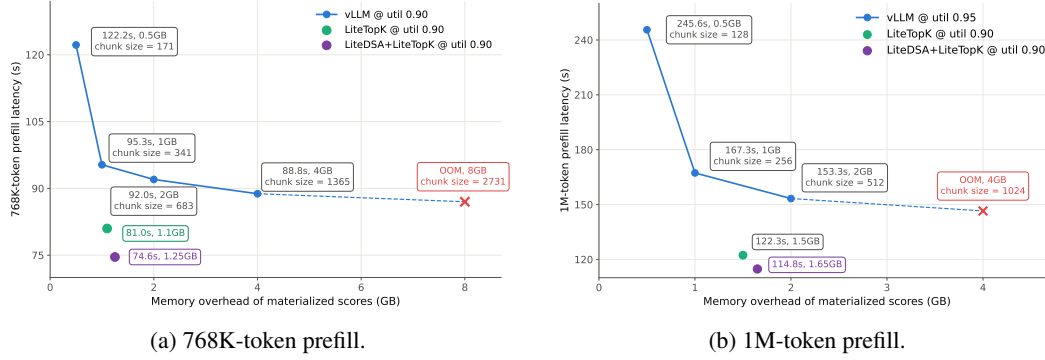


Figure 4: Memory-overhead vs End2End latency, GLM-5.2-FP8, 8xB200, TP8

End-to-End Long-Context Prefilling. We first evaluate LITETOPK and LITEDSA in end-to-end deployments of GLM-5.2 (GLM-5-Team et al., 2026) and LongCat 2.0 Meituan LongCat Team (2026). Both models natively support one-million-token contexts, and we evaluate their prefill stages at context lengths of 768K and 1M tokens. We serve the FP8 model using vLLM 0.23 (Kwon et al., 2023) on eight NVIDIA B200 GPUs and replace the original DSA kernel with LiteTopK only for context lengths starting from 128K and the LITEDSA is used from 4K, while leaving the rest of the serving stack unchanged to isolate the impact of our method. Following the official vLLM configuration, we enable eight-way tensor parallelism and expert parallelism, compile the model with `torch.compile` and CUDA graphs, enable asynchronous scheduling to emulate realistic serving workloads, and set the prefill chunk size to 8,192 tokens.

For the 1M-token prefill setting, the default configuration with `gpu_memory_utilization` set to 0.90 results in an out-of-memory error, so we follow the recommended configuration and increase it to 0.95. For the 768K-token prefill setting, we keep `gpu_memory_utilization` at 0.90. For the baseline, which materializes intermediate score logits, we control memory consumption through additional prefill sub-chunking. Specifically, we vary the maximum score-logit footprint from 512 MB to 1 GB, 2 GB, and 4 GB. A larger budget reduces the amount of sub-chunking and can reduce latency, but also increases peak memory consumption and eventually causes the baseline to run out of memory. LiteTopK, in contrast, avoids materializing these large intermediate score logits and can therefore process the full 8,192-token prefill chunk without additional sub-chunking. We report end-to-end prefill latency and peak auxiliary memory consumption to characterize this latency-memory trade-off.

Sparse-Attention Kernel Benchmark. We next isolate the DSA workload to examine the source of the end-to-end improvements. We compare LiteTopK against two implementations. The first is the official DSA implementation³, which uses a custom CUDA kernel for score computation and invokes PyTorch’s `topk` operator for Top- k selection. The second, denoted as vLLM, is an optimized implementation for the NVIDIA Blackwell architecture that replaces PyTorch’s `topk` operator to reduce unnecessary I/O traffic. We also compare LiteDSA with its official attention kernel.

We construct the benchmark inputs from the first-layer attention activations of GLM-5.2, using text sampled from Wikipedia (Merity et al., 2016). We consider context lengths of 256K, 512K, 768K, and 1M tokens, vary the prefill chunk size from 128 to 8,192 tokens, and fix k to 2,048. For each configuration, we report the aggregate kernel latency for processing 8,192 tokens. When the queries are divided into multiple chunks, the reported latency is the sum of the latency across all chunks. We additionally measure peak auxiliary memory consumption. For chunk sizes of 2,048, 4,096, and 8,192, both the official DSA and vLLM baselines require up to 8 GB, 16 GB, and 32 GB of additional memory, respectively. Although such memory overheads are impractical for production deployment, we retain these configurations to characterize the best latency achievable by the baselines when sufficient memory is available.

Large-Scale Retrieval Benchmark. Finally, we evaluate LiteTopK on large-scale vector retrieval to determine whether its efficiency gains generalize beyond sparse attention. We construct a corpus

³<https://github.com/deepseek-ai/DeepGEMM>

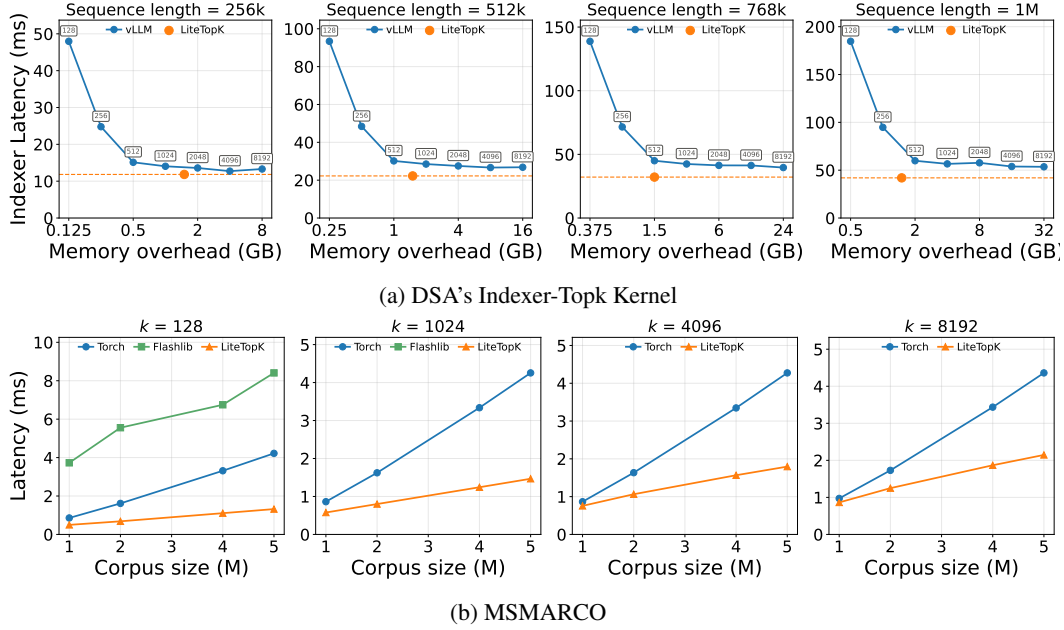


Figure 5: Runtime comparison between LiteTopK and existing methods on sparse-attention and retrieval workloads.

of five million passages sampled from MSMARCO-V2.1⁴, a widely used text-retrieval dataset. Each passage is encoded as a 768-dimensional vector using the Snowflake Arctic-embed-m-v1.5 model (Merrick, 2024). We construct the query set Q by randomly sampling 1,000 passages and evaluate corpus sizes of 1M, 2M, 4M, and 5M vectors. We vary k over 128, 1,024, 4,096, and 8,192 and use a batch size of 64 to emulate concurrent retrieval requests. Because the DSA and vLLM baselines above are specialized for attention workloads, we instead compare LiteTopK with Flashlib and Torch on this task and report Flashlib results for the configurations it supports.

Evaluation Metrics. LiteTopK preserves the input and output semantics of the original Top- k implementations. Therefore, our evaluation focuses on efficiency. For the end-to-end serving experiment, we report prefill latency and peak auxiliary memory consumption. For the isolated sparse-attention and retrieval benchmarks, we report aggregate kernel latency and peak auxiliary memory consumption under each workload configuration.

Implementation. All our implementations are written in CUDA C++ using CuTe/CUTLASS, while the baselines are evaluated base on their official implementations. The experiments are conducted on an NVIDIA B200 GPU with 179 GB of memory using CUDA 12.8, and we additionally conduct experiments on H100 GPUs. All reported running times are averaged over 20 runs.

4.2 EXPERIMENTAL RESULTS

Results Overview. We first evaluate the end-to-end benefits of LiteTopK in long-context model serving, and then analyze the underlying kernel-level improvements and their portability across GPU architectures. Finally, we evaluate LiteTopK on large-scale retrieval to examine whether its advantages generalize beyond sparse-attention workloads.

End-to-End Long-Context Prefilling. Figure 4 reports the end-to-end prefill latency and peak auxiliary memory consumption of GLM-5.2 in a realistic serving configuration. The vLLM baseline reduces latency by increasing its internal prefill sub-chunk size, but this improvement comes at the cost of substantially higher memory consumption and eventually results in an out-of-memory error. Its fastest feasible configuration uses a sub-chunk size of 512 and requires 153.3 s to process

⁴<https://huggingface.co/datasets/Snowflake/msmarco-v2.1-snowflake-arctic-embed-m-v1.5>

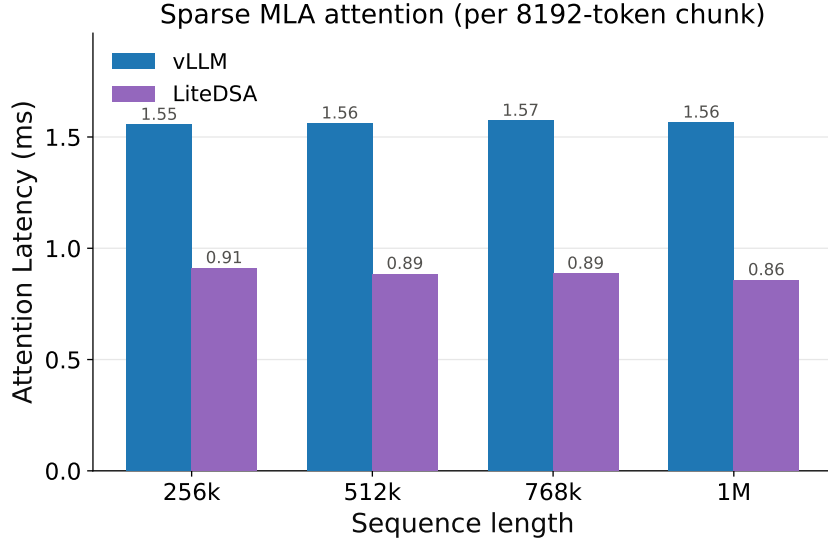


Figure 6: Runtime comparison between official DSA Attention and the LiteDSA.

the 1M-token input, with 2.0 GB of auxiliary memory. LiteTopK completes the same workload in 128.4 s while consuming only 1.5 GB, corresponding to a $1.26\times$ speedup with 25% lower auxiliary memory consumption. LITEDSA further increases the speedup to $1.34\times$ while introducing negligible additional memory overhead.

The reported memory consumption of LiteTopK includes a deliberately conservative candidate buffer with a capacity of $12k$, corresponding to 24,576 candidates for $k = 2,048$. In practice, the number of candidates retained after filtering is typically substantially smaller than this capacity. The reported 1.5 GB footprint therefore represents a conservative implementation point and can be further reduced through tighter candidate-buffer allocation. These results demonstrate that avoiding score-logit materialization allows LiteTopK to translate its kernel-level efficiency into measurable end-to-end latency and memory improvements.

Sparse-Attention Indexer-Topk Kernel Performance. Figure 5a compares LiteTopK with the official DSA implementation and the optimized vLLM kernel on NVIDIA B200 GPUs. Under the same chunk size of 8,192 and a context length of 1M, LiteTopK reduces the aggregate kernel latency from 146.6 ms to 41.9 ms relative to DSA, yielding a $3.5\times$ speedup. It also outperforms the Blackwell-optimized vLLM implementation by $1.24\times$, reducing latency from 52.3 ms to 41.9 ms.

This equal-chunk-size comparison isolates the computational efficiency of the kernels, but the baseline configurations require 32 GB of auxiliary memory at a chunk size of 8,192, making them difficult to use in memory-constrained deployments. When vLLM instead uses more practical chunk sizes of 512 and 1,024, its latency increases to 59.8 ms and 55.6 ms, respectively. Because LiteTopK has substantially lower memory overhead, it can retain the 8,192-token chunk size and achieve deployment-oriented speedups of $1.38\times$ and $1.28\times$ over these two vLLM configurations.

The advantage of LiteTopK also increases with context length. At a chunk size of 8,192, LiteTopK reduces the DSA latency from 36.6 ms to 12.6 ms at a context length of 256K, corresponding to a $2.90\times$ speedup. At a context length of 1M, the speedup increases to $3.38\times$. This widening performance gap indicates that LiteTopK scales more favorably as the number of candidate tokens grows.

Sparse-Attention Attention Kernel Performance. As shown in Figure 6, our proposed LiteDSA achieves a $1.7\times$ - $1.81\times$ speedup over the official attention kernel across different sequence lengths. This is because DSA performs attention only over the selected top-2048 candidates rather than the full KV sequence, making its runtime much less sensitive to sequence length.

Portability to NVIDIA H100. We further evaluate LiteTopK on NVIDIA H100 GPUs, as shown in Figure 7, while fixing the prefill chunk size to 8,192 tokens. The vLLM baseline evaluated above

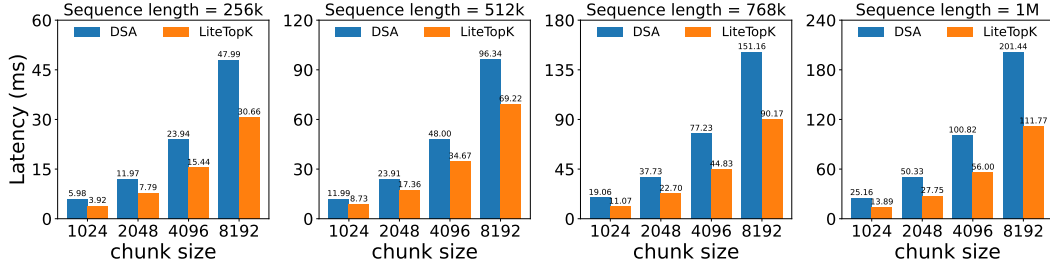


Figure 7: Runtime comparison between LiteTopK and the official DSA implementation on NVIDIA H100 GPUs. The prefill chunk size is fixed to 8,192 tokens.

relies on optimizations specific to the Blackwell architecture and is therefore unavailable on H100; accordingly, we compare LiteTopK with the official DSA implementation. LiteTopK retains a similar acceleration trend across the evaluated context lengths, indicating that its benefits are not limited to Blackwell GPUs.

Large-Scale Retrieval Performance. Figure 5b reports the results on the MSMARCO retrieval workload. LiteTopK consistently outperforms FlashLib across the evaluated values of k . Even at $k = 128$, FlashLib is substantially slower than LiteTopK, and the performance gap widens further as k increases. This degradation is consistent with the increasing cost of maintaining FlashLib’s ordered on-chip data structure: as k grows, both the size of the maintained state and the cost of updating it increase. Consequently, FlashLib is better suited to very small- k workloads, whereas LiteTopK remains efficient for the large- k configurations required by our sparse-attention and retrieval workloads.

5 CONCLUSION AND DISCUSSION

This paper proposes an Indexer–TopK fused operator to accelerate the prefill stage of sparse attention in long-context inference. A promising direction for future work is to extend this approach to the decode stage, particularly by integrating it with speculative decoding.

REFERENCES

- Tolu Alabi, Jeffrey D Blanchard, Bradley Gordon, and Russel Steinbach. Fast k-selection algorithms for graphics processing units. *Journal of Experimental Algorithmics (JEA)*, 17:4–1, 2012.
- Yushi Bai, Qian Dong, Ting Jiang, Xin Lv, Zhengxiao Du, Aohan Zeng, Jie Tang, and Juanzi Li. Index-cache: Accelerating sparse attention via cross-layer index reuse. *arXiv preprint arXiv:2603.12201*, 2026.
- Sebastian Bruch. *Foundations of vector retrieval*, volume 1. Springer, 2024.
- Renze Chen, Zhuofeng Wang, BeiQuan Cao, Tong Wu, Size Zheng, Xiuhong Li, Xuechao Wei, Shengen Yan, Meng Li, and Yun Liang. Arkvale: Efficient generative llm inference with recallable key-value eviction. *Advances in Neural Information Processing Systems*, 37:113134–113155, 2024.
- Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35: 16344–16359, 2022.
- DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026.
- Weihao Gao, Xiangjun Fan, Chong Wang, Jiankai Sun, Kai Jia, Wenzhi Xiao, Ruofan Ding, Xingyan Bin, Hui Yang, and Xiaobing Liu. Learning an end-to-end structure for retrieval in large-scale recommendations. In *Proceedings of the 30th ACM international conference on information & knowledge management*, pp. 524–533, 2021.

GLM-5-Team, :, Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, Chenzheng Zhu, Congfeng Yin, Cunxiang Wang, Gengzheng Pan, Hao Zeng, Haoke Zhang, Haoran Wang, Huilong Chen, Jiajie Zhang, Jian Jiao, Jiaqi Guo, Jingsen Wang, Jingzhao Du, Jinzhu Wu, Kedong Wang, Lei Li, Lin Fan, Lucen Zhong, Mingdao Liu, Mingming Zhao, Pengfan Du, Qian Dong, Rui Lu, Shuang-Li, Shulin Cao, Song Liu, Ting Jiang, Xiaodong Chen, Xiaohan Zhang, Xuancheng Huang, Xuezheng Dong, Yabo Xu, Yao Wei, Yifan An, Yilin Niu, Yitong Zhu, Yuanhao Wen, Yukuo Cen, Yushi Bai, Zhongpei Qiao, Zihan Wang, Zikang Wang, Zilin Zhu, Ziqiang Liu, Zixuan Li, Bojie Wang, Bosi Wen, Can Huang, Changpeng Cai, Chao Yu, Chen Li, Chengwei Hu, Chenhui Zhang, Dan Zhang, Daoyan Lin, Dayong Yang, Di Wang, Ding Ai, Erle Zhu, Fangzhou Yi, Feiyu Chen, Guohong Wen, Hailong Sun, Haisha Zhao, Haiyi Hu, Hanchen Zhang, Hanrui Liu, Hanyu Zhang, Hao Peng, Hao Tai, Haobo Zhang, He Liu, Hongwei Wang, Hongxi Yan, Hongyu Ge, Huan Liu, Huanpeng Chu, Jia'ni Zhao, Jiachen Wang, Jiajing Zhao, Jiamin Ren, Jiapeng Wang, Jiaxin Zhang, Jiayi Gui, Jiayue Zhao, Jijie Li, Jing An, Jing Li, Jingwei Yuan, Jinhua Du, Jinxin Liu, Junkai Zhi, Junwen Duan, Kaiyue Zhou, Kangjian Wei, Ke Wang, Keyun Luo, Laiqiang Zhang, Leigang Sha, Liang Xu, Lindong Wu, Lintao Ding, Lu Chen, Minghao Li, Nianyi Lin, Pan Ta, Qiang Zou, Rongjun Song, Ruiqi Yang, Shangqing Tu, Shangtong Yang, Shaoxiang Wu, Shengyan Zhang, Shijie Li, Shuang Li, Shuyi Fan, Wei Qin, Wei Tian, Weining Zhang, Wenbo Yu, Wenjie Liang, Xiang Kuang, Xiangmeng Cheng, Xiangyang Li, Xiaoquan Yan, Xiaowei Hu, Xiaoying Ling, Xing Fan, Xingye Xia, Xinyuan Zhang, Xinze Zhang, Xirui Pan, Xu Zou, Xunkai Zhang, Yadi Liu, Yandong Wu, Yanfu Li, Yidong Wang, Yifan Zhu, Yijun Tan, Yilin Zhou, Yiming Pan, Ying Zhang, Yinpei Su, Yipeng Geng, Yong Yan, Yonglin Tan, Yuean Bi, Yuhao Shen, Yuhao Yang, Yujiang Li, Yunan Liu, Yunqing Wang, Yuntao Li, Yurong Wu, Yutao Zhang, Yuxi Duan, Yuxuan Zhang, Zezhen Liu, Zhengtao Jiang, Zhenhe Yan, Zheyu Zhang, Zhixiang Wei, Zhuo Chen, Zhuoer Feng, Zijun Yao, Ziwei Chai, Ziyuan Wang, Zuzhou Zhang, Bin Xu, Minlie Huang, Hongning Wang, Juanzi Li, Yuxiao Dong, and Jie Tang. Glm-5: from vibe coding to agentic engineering, 2026. URL <https://arxiv.org/abs/2602.15763>.

Ankit Gupta, Guy Dar, Shaya Goodman, David Ciprut, and Jonathan Berant. Memory-efficient transformers via top-k attention. In *Proceedings of the Second Workshop on Simple and Efficient Natural Language Processing*, pp. 39–52, 2021.

Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Kangdi Chen, Yuhao Dong, and Yu Wang. Flashdecoding++: Faster large language model inference with asynchronization, flat gemm optimization, and heuristics. *Proceedings of Machine Learning and Systems*, 6:148–161, 2024.

Piotr Indyk and Rajeev Motwani. Approximate nearest neighbors: Towards removing the curse of dimensionality. In *Proceedings of the Thirtieth Annual ACM Symposium on the Theory of Computing, Dallas, Texas, USA, May 23-26, 1998*, pp. 604–613, 1998.

Sujay Khandagale, Bhawna Juneja, Prabhat Agarwal, Aditya Subramanian, Jaewon Yang, and Yuting Wang. Interactrank: Personalized web-scale search pre-ranking with cross interaction features. In *Companion Proceedings of the ACM on Web Conference 2025, WWW 2025, Sydney, NSW, Australia, 28 April 2025 - 2 May 2025*, pp. 287–295, 2025.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pp. 611–626, 2023.

Xunhao Lai, Weiqi Xu, Yufeng Yang, Qiaorui Chen, Yang Xu, Lunbin Zeng, Xiaolong Li, Haohai Sun, Haichao Zhu, Vito Zhang, et al. Minimax sparse attention. *arXiv preprint arXiv:2606.13392*, 2026.

Jinhyuk Lee, Zhuoyun Dai, Sai Meher Karthik Duddu, Tao Lei, Iftekhar Naim, Ming-Wei Chang, and Vincent Zhao. Rethinking the role of token retrieval in multi-vector retrieval. *Advances in Neural Information Processing Systems*, 36:15384–15405, 2023.

Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation. *Advances in Neural Information Processing Systems*, 37:22947–22970, 2024.

- Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, et al. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.
- Meituan LongCat Team. Introducing longcat-2.0. <https://longcat.ai/blog/longcat-2.0>, June 2026. GitHub: <https://github.com/meituan-longcat/LongCat-2.0>.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016.
- Luke Merrick. Embedding and clustering your data can improve contrastive pretraining. *arXiv preprint arXiv:2407.18887*, 2024.
- Piotr Nawrot, Jianing Li, Renjie Huang, Sebastian Ruder, Kelly Marchisio, and Edoardo Maria Ponti. The sparse frontier: Sparse attention trade-offs in transformer llms. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 38667–38701, 2026.
- Luka Ribar, Ivan Chelombiev, Luke Hudlass-Galley, Charlie Blake, Carlo Luschi, and Douglas Orr. Sparq attention: Bandwidth-efficient LLM inference. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*, volume 235 of *Proceedings of Machine Learning Research*, pp. 42558–42583, 2024.
- Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. *Advances in Neural Information Processing Systems*, 37:68658–68685, 2024.
- Ryan Synk, Monte Hoover, John Kirchenbauer, Neel Jain, Alex Stein, Manli Shu, Josue Melendez Sanchez, Ramani Duraiswami, and Tom Goldstein. Exploiting sparsity for long context inference: Million token contexts on commodity gpus. *arXiv preprint arXiv:2502.06766*, 2025.
- Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context llm inference. *arXiv preprint arXiv:2406.10774*, 2024.
- Roger Weber, Hans-Jörg Schek, and Stephen Blott. A quantitative analysis and performance study for similarity-search methods in high-dimensional spaces. In *VLDB’98, Proceedings of 24rd International Conference on Very Large Data Bases, August 24-27, 1998, New York City, New York, USA*, pp. 194–205, 1998.
- Lijie Yang, Zhihao Zhang, Zhuofu Chen, Zikun Li, and Zhihao Jia. Tidaldecode: Fast and accurate LLM decoding with position persistent sparse attention. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*, 2025a.
- Lijie Yang, Zhihao Zhang, Arti Jain, Shijie Cao, Baihong Yuan, Yiwei Chen, Zhihao Jia, and Ravi Netravali. Less is more: Training-free sparse attention with global locality for efficient reasoning. *arXiv preprint arXiv:2508.07101*, 2025b.
- Shuo Yang, Haocheng Xi, Yilong Zhao, Qiuyang Mang, Zhe Wang, Shanlin Sun, Kurt Keutzer, Joseph E. Gonzalez, Song Han, Chenfeng Xu, and Ion Stoica. Flashlib: Bringing flash magic to classical machine learning operators, 2026. URL <https://flashml-org.github.io/>.
- Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, et al. Flashinfer: Efficient and customizable attention engine for llm inference serving. *Proceedings of Machine Learning and Systems*, 7, 2025.
- Ziqi Yin, Gao Cong, Kai Zeng, Jinwei Zhu, and Bin Cui. Bbc: Improving large-k approximate nearest neighbor search with a bucket-based result collector. *arXiv preprint arXiv:2604.01960*, 2026.
- Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Yuxing Wei, Lean Wang, Zhiping Xiao, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 23078–23097, 2025.

Hailin Zhang, Xiaodong Ji, Yilin Chen, Fangcheng Fu, Xupeng Miao, Xiaonan Nie, Weipeng Chen, and Bin Cui. Pqcache: Product quantization-based kvcache for long context llm inference. *Proceedings of the ACM on Management of Data*, 3(3):1–30, 2025.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.